Data Transfers: Securely Linking Your Local Machine with GitHub

Getting Started with Git and GitHub Part 1
Coffee, Cookies, and Coding (C-cubed) Workshops

September 29, 2025

Once you have Git installed and a GitHub account set up, you need to configure them to securely and accurately transfer information between your local devices and your remote repository on GitHub.

Configure Git

Git associates a username and email address with the project versions you create. You have the option to configure these settings globally, applying one user profile to all repositories, or locally, applying a user profile to a specific repository. A local setting will override the global setting [1,2].

Let's begin by checking the current configurations on your system. Using `--list` `--show-origin` instead will display the file location for all configurations [3].

Command-Line Interface

```
git config --list
```

Output - Mac

```
gpg.format=ssh                                # SSH keys used for signing
commit.gpgsign=true                           # Sign all GitHub actions
filter.lfs.clean=git-lfs clean -- %f          # Cleanly stores large files
filter.lfs.smudge=git-lfs smudge -- %f        # Retrieves the large files
filter.lfs.process=git-lfs filter-process     # Seamlessly manage large files
filter.lfs.required=true                      # List files in .gitattributes
init.defaultbranch=main                       # Default branch name is "main"
credential.helper=osxkeychain                 # Store credentials in Keychain
user.name=Shelby Golden                       # Username for commits
user.email=shelby.golden@yale.edu             # Email for commits
pack.packsizelimit=50M                        # Sets size limit for pac files
```

Output - PC

```
http.sslbackend=schannel                      # SSL backend for HTTPS
diff.astexplain.texconv=astextplain           # Diffs as "astextplain"
filter.lfs.clean=git-lfs clean -- %f          # Cleanly stores large files
filter.lfs.smudge=git-lfs smudge -- %f        # Retrieves the large files
filter.lfs.process=git-lfs filter-process     # Seamlessly manage large files
filter.lfs.required=true                      # List files in .gitattributes
core.autocrlf=true                            # Standardize line ends
core.fscache=true                             # Enable filesystem caching
core.symlinks=false                           # Disables symbolic links
pull.rebase=false                             # Use merge when pulling
credential.helper=manager                     # Credential manager for PC
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=main                       # Default branch name is "main"
user.name=Shelby Golden                       # Username for commits
user.email=shelby.golden@yale.edu             # Email for commits
```

The results above are examples of potential configurations you might see on your system. In this example, the `user.name` and `user.email` settings are already configured.

If you do not see these configurations in your results, you will need to set them up on your system. We suggest you start by setting the global credential configurations. Use `--local` instead if you wish to apply the configurations to the currently open directory in your command-line interface.

Command-Line Interface

```
git config --global user.name "Your Name Here"
git config --global user.email "your@email.com"
```

If you make a mistake, you can use the `--unset` command to remove the configurations through the terminal, or you can modify the .gitconfig file directly [4].

Command-Line Interface

```
git config --global --unset user.name
git config --global --unset user.email
```

It is possible to automatically manage multiple credentials without needing to override the global settings in each folder. This is discussed in the **Appendix**.

Transfer Protocols

GitHub supports two types of authentication and file transfer protocols: Hypertext Transfer Protocol Secure (HTTPS) and Secure Shell (SSH) Keys. These protocols are used to securely and accurately communicate project access levels, rules, and, most importantly, to transfer files between your local device and the remote repository [5].

Below is a list of pros and cons for both [6–9]:

Criteria	SSH	HTTPS
Setup	More involved.	Easier for beginners.
Ease of Use	Low maintenance.	Requires username and PAT with each interaction.

Criteria	SSH	HTTPS
Security	More secure.	Vulnerable to brute force attacks.
Permissions Control	Fine- and coarse-grained options.	Coarser permissions for PAT set by GitHub.
Signing	SSH Keys and GNU Privacy Guard (GPG)	GPG and S/MIME
Other	Additional security with Hardware Keys [10].	Never blocked by firewalls and proxies [5].

! PATs Replaced Username/Password

Previously, GitHub allowed authentication with a username and password, but in 2021 the password requirement was replaced with PATs. Credential managers were adjusted to accommodate this change and should store the PAT accordingly.

Code Like a Pro

You can use different protocols for different projects, and it is possible to change the protocol associated with a given project later. Directions for converting between the two transfer methods can be found in the *Managing remote repositories* article provided on docs.github.com [11,12].

We provide basic instructions to configure your Git and GitHub for both methods. Advanced settings are not covered here, but relevant links are sometimes provided to help with these setups. Users are encouraged to explore these resources as needed for their individual projects.

HTTPS

These directions were provided in the *Managing your personal access tokens* article provided on docs.github.com [9].

1. Go to your GitHub account settings page: https://github.com/settings/profile.
2. At the end of the left sidebar, click on "<> Developer Settings".

3. In the left sidebar open the "🔑 Personal access tokens" drop-down menu and click on "Tokens (classic)".

4. In the center of the window, there is a drop-down menu. Select "Generate new token (classic)".

5. Add a note describing what the PAT is for and set its expiration time-frame.

> **Set an Expiration**
>
> You have the option to select "No expiration," but this option is strongly discouraged for security purposes.

6. In the "Select scopes" section, check "repo" for full control of private repositories.

> **Fine-Tuning Scopes**
>
> We do not cover the different ways repository access can be fine-tuned. If you wish to learn more about the access permission levels possible through GitHub, you can read the *Scopes for OAuth apps* article provided on docs.github.com [@oath].

7. At the bottom of the page, click Generate token.

8. Copy the generated PAT and save it somewhere secure for later use, as you will not be able to access it again after you exit the window.

Every time you interact with GitHub, you will enter your PAT instead of your account password. For example:

Command-Line Interface

```
git clone https://github.com/USERNAME/REPO.git

Username: YOUR-USERNAME
Password: YOUR-PERSONAL-ACCESS-TOKEN
```

SSH Keys

SSH Keys work by creating a private and public key pair. When interacting with the remote repository, the keys are compared to determine access permissions. We will use the command-line interface to generate these keys, and you will upload your public key to GitHub to complete the process [8,13,14].

It is best practice to review your system for any existing private/public key pairs.

Command-Line Interface

```
ls -al ~/.ssh          # List all elements in the .ssh directory
```

Output - SSH Keys Exist

```
.                               # Unix-like reference to current directory
..                              # Unix-like reference to parent directory
.DS_Store                       # MacOS folder custom attributes metadata
config                          # SSH Key configurations
id_ed25519                      # SSH authentication local key
id_ed25519.pub                  # SSH authentication public key
id_ed25519_signing              # SSH signing local key
id_ed25519_signing.pub          # SSH signing public key
known_hosts                     # GitHub's SSH Key fingerprints
```

Output - No Previous Keys Available

```
ls: cannot access `/c/Users/userID/.ssh`: No such file or directory
```

In this example, you can see that I have two SSH key pairs: one for authentication and one for signing. They are configured to use the Edwards-curve Digital Signature Algorithm (`id_ed25519`), a type of public-key cryptography algorithm. You might also see Rivest-Shamir-Adleman (`id_rsa`) or Elliptic Curve Digital Signature Algorithm (`id_ecdsa`) [15,16].

While RSA is widely used and is compatible with SHA-1 and SHA-2 hash algorithms, ED25519 is considered to be more efficient, faster, and more secure because of its elliptic curve design. It is also a more modern method for cryptography compared to RSA [16].

We will show you how to generate an `id_ed25519` or `id_rsa` SSH key. Consider the points provided below when choosing an algorithm.

! DSA Keys No Longer Supported

As of March 15, 2022, GitHub no longer supports the `ssh-dss` algorithm. If your setup uses this algorithm, you will need to delete those keys and replace them with either `id_rsa` or `id_ed25519` [8].

! Must Specify SHA-2 Hash Length for RSA

SSH keys generated using the RSA algorithm after November 2, 2021, must be specified to use the SHA-2 hash algorithm length. RSA keys with a valid_after date set before this date can use either SHA-1 or SHA-2 [8,15].

You can verify the key length with `ssh-keygen -l -f ~/.ssh/id_rsa.pub`, changing the file name as needed. If the length is 2048, the key is using SHA-1. If the length is 4096, it is using SHA-2 [15].

We suggest replacing any SHA-1 configured keys with SHA-2 configured keys.

! Authentication vs Signing SSH Keys

GitHub requires different SSH Keys for account authentication and signing, which verify the authenticity and integrity of GitHub actions [17]. At a minimum, you will need keys for authentication, while signing is optional. The same setup is applied for both options, except when specifying the purpose of the key to GitHub.

Code Like a Pro

The astute student might have observed that my system is set to automatically sign all actions in GitHub with `commit.gpgsign=true` in the .gitconfig file. If you choose to create a pair of SSH keys for signing, it is recommended that you add this to your configuration with: `git config commit.gpgsign true`.

The GPG Suite for macOS and Gpg4win for Windows will allow you to configure the GPG agent and save your signing credentials. You can learn more about signing commits in the *About commit signature verification* and *Signing commits* articles provided on

docs.github.com [17–19].

Now that you have chosen an SSH algorithm from ED25519 or RSA, we can generate the public and private key pair.

Command-Line Interface

```
# Ed25519
ssh-keygen -t ed25519 -C "your@email.com"

# RSA with SHA-2
ssh-keygen -t rsa -b 4096 -C "your@email.com"
```

Output - ED25519

```
Generating public/private ed25519 key pair.
Enter a file in which to save the key (/Users/userID/.ssh/id_ed25519):
```

Output - RSA

```
Generating public/private rsa key pair.
Enter a file in which to save the key (/Users/userID/.ssh/id_rsa):
```

After the keys have been created, you are given the option to specify where they will be stored, and you can modify the key name if desired. It is easier to keep the SSH key pairs stored in the ~/.ssh directory, avoiding additional configuration settings.

If you want to change the key name, you **must** keep the `id_ed25519` or `id_rsa` prefix, as this tells your system which algorithm to use. For example:

Output - ED25519

```
# Hit enter to accept the default location and file name
Enter a file in which to save the key (/Users/userID/.ssh/id_ed25519):
```

```
# Specify a new file name, keeping the default location
Enter a file in which to save the key (/Users/userID/.ssh/id_ed25519):
↪  /Users/userID/.ssh/id_ed25519_modify_name
```

Continuing with the ED25519 example, as the steps will be the same for RSA going forward, you will be given the option to add a password to your key. We highly suggest that you use a strong password and save it somewhere secure for later use. If needed, you can change the password down the line, but you will not be able to recover it if you lose it [13]

Output - ED25519

```
Enter passphrase for "/Users/userID/.ssh/id_ed25519" (empty for no
↪  passphrase):
```

```
Enter same passphrase again:
```

Output - ED25519

```
Your identification has been saved in /Users/userID/.ssh/id_ed25519
Your public key has been saved in /Users/userID/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:nmQLX/8Tew2X1ju3sOb4wSNK+HIycTOLKEsPzYfN+Ms shelby.golden@yale.edu
The key's randomart image is:
+--[ED25519 256]--+
|                 |
|                 |
|                 |
|                 |
|      + S o     o|
|     + &.X +. ooo|
|    . =.X...+=.=o|
|     . =+o..ooBo+|
|        =E..++.++|
+----[SHA256]-----+
```

When you have successfully created your keys, you will see the final output shown above. It is not necessary to save the fingerprint or randomart (visual representation of the fingerprint), but I personally save them in the notes section of my password manager.

Similarly to HTTPS, it is possible to cache your SSH key password using agent forwarding. While we won't cover the setup process here, you can learn more about it and find detailed instructions in the *Using SSH agent forwarding* and *Generating a new SSH key and adding it to the ssh-agent* articles on docs.github.com [20,21].

With our public and private SSH key pairs created, we are ready to add the public one to our GitHub account. Start by copying the recently created public key.

Command-Line Interface - ED25519

```
pbcopy < ~/.ssh/id_ed25519.pub
```

1. Go to your GitHub account settings page: https://github.com/settings/profile.
2. In the "Access" section of the left sidebar, click on "🔑 SSH and GPG Keys".
3. Click the New SSH key button in the top right.
4. Add a title explaining the use of the key (e.g., Personal laptop).
5. Select "Authentication key" as the key type.
6. In the "Key" field, paste the public key you copied above.
7. Click Add SSH key and follow the prompts.
8. To ensure a valid connection with GitHub, you will need to open the ~/.ssh/known_hosts file and copy-and-paste GitHub's SSH key fingerprints from https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/githubs-ssh-key-fingerprints [22].

You can confirm that the keys are correctly configured with the following test. If you get the expected output shown below, then you are all set!

Command-Line Interface

```
ssh -T git@github.com
```

Output

```
Hi USERNAME! You've successfully authenticated, but GitHub does not provide
↪  shell access.
```

Appendix

Multiple Git Configurations

Suppose you use two or more GitHub accounts and need different emails or usernames associated with Git commits made to those remote repositories. Reconfiguring every project directory for this purpose would be tedious.

Instead, you can use an automatically expanded local setting to override the global configurations using "Conditional Includes". This feature has been available since Git version 2.13 [23–25].

In this framework, you need to organize your project directories such that those with an alternate configuration are in the same parent directory. We will refer to this parent directory as "Alternate." The directions given here assume that you have already set up the global Git configurations.

We start by locating and opening the .gitconfig file.

Command-Line Interface

```
git config --list --show-origin
```

Abbreviated Output

```
:
file:/Users/userID/.gitconfig   user.email=shelby.golden@yale.edu
file:/Users/userID/.gitconfig   user.name=Shelby Golden
:
```

When you open .gitconfig with Finder/File Explorer, you will see something like this:

/Users/userID/.gitconfig

```
[user]
        email = shelby.golden@yale.edu
        name = Shelby Golden
        signingkey = /Users/userID/.ssh/id_ed25519_jhu_signing.pub
[gpg]
        format = ssh
[commit]
        gpgsign = true
[init]
        defaultBranch = main
[filter "lfs"]
        required = true
        clean = git-lfs clean -- %f
        smudge = git-lfs smudge -- %f
        process = git-lfs filter-process
[pack]
        packSizeLimit = 50M
```

After the [user] section, add the conditional include statement. The order of the sections does not matter, but the indentation is important to keep consistent.

/Users/userID/.gitconfig

```
[user]
        email = shelby.golden@yale.edu
        name = Shelby Golden
        signingkey = /Users/userID/.ssh/id_ed25519_jhu_signing.pub
[includeIf "gitdir:/Users/userID/Desktop/.../Alternate/"]
        path = /Users/userID/Desktop/DSDE/.../Alternate/.gitconfig
:
```

Now we need to create the Git configuration file applicable to all projects stored in ~/Alternate.

Command-Line Interface

```
# 1. Navigate to the alternative account parent directory.
cd "/FILE_PATH/Alternate"

# 2. Create a new configuration file.
touch .gitconfig
```

Open the ~/Alternate/.gitconfig file, either using the `vim` in the command-line interface or by opening it with Finder/File Explorer, and add the relevant Git configurations you want applied to this account.

/Alternate/.gitconfig

```
[user]
        email = shelby.golden@gmail.com
        signingkey = /Users/userID/.ssh/id_ed25519_personal_signing.pub
[gpg]
        formal = ssh
[commit]
        gpgsign = true
:
```

Configuring SSH Keys to Different Hosts

Suppose you want to use SSH Keys to interact with remote repositories stored on different domains, such as GitHub and GitLab. You will need to create a new document called config in the ~/.ssh directory.

Command-Line Interface

```
# 1. Navigate to the .ssh directory.
cd /Users/userID/.ssh

# 2. Create a new configuration file.
touch config
```

Open the ~/.ssh/config file, either using `vim` in the command-line interface or by opening it with Finder/File Explorer, and add the relevant SSH configurations necessary to coordinate file transfers between different remote account locations.

For example, my configuration document looks like this:

/.ssh/config

```
Host github.com
        User git
        AddKeysToAgent yes
        UseKeychain yes
        IdentityFile ~/.ssh/id_ed25519_github
Host gitlab.com
        User git
        AddKeysToAgent yes
        IdentityFile ~/.ssh/id_ed25519_gitlab
```

References

1. Atlassian. Git config. *Atlassian Tutorial*.
2. Git-SCM. Git config. *Git-SCM Documents*.

3. kgf3JfUtW, A. by. How do i show my global git configuration? - stack overflow. *StackOverflow* (2017).
4. Foster, G. Unsetting git configuration settings. *Graphite* (2024).
5. GitHub. About authentication to GitHub. *GitHub Docs*.
6. GitHub. Set up git - authenticating with GitHub from git. *GitHub Docs*.
7. Singh, Y. Difference between SSH and HTTPS in GitHub. *Medium* (2023).
8. GitHub. Generating a new SSH key and adding it to the ssh-agent - generating a new SSH key. *GitHub Docs*.
9. GitHub. Managing your personal access tokens. *GitHub Docs*.
10. GitHub. Generating a new SSH key and adding it to the ssh-agent - generating a new SSH key for a hardware security key. *GitHub Docs*.
11. GitHub. Managing remote repositories - switching remote URLs from SSH to HTTPS. *GitHub Docs*.
12. GitHub. Managing remote repositories - switching remote URLs from HTTPS to SSH. *GitHub Docs*.
13. GitHub. Working with SSH key passphrases. *GitHub Docs*.
14. GitHub. Adding a new SSH key to your GitHub account. *GitHub Docs*.
15. StrongDM, S. T. Z. T. P. A. M. (PAM). Comparing SSH keys: A comprehensive guide (RSA, DSA, ECDSA). (2025).
16. GeeksforGeeks. RSA vs Ed25519: Which key pair is right for your security needs? *GeeksforGeeks* (2025).
17. GitHub. About commit signature verification - SSH commit signature verification. *GitHub Docs*.
18. GitHub. Signing commits. *GitHub Docs*.
19. GitHub. Signing tags. *GitHub Docs*.
20. GitHub. Using SSH agent forwarding. *GitHub Docs*.
21. GitHub. Generating a new SSH key and adding it to the ssh-agent - adding your SSH key to the ssh-agent. *GitHub Docs*.
22. GitHub. GitHub's SSH key fingerprints. *GitHub Docs*.
23. Janousek, T. A. by. Git - can i specify multiple users for myself in .gitconfig? *StackOverflow* (2017).
24. Barochiya, D. How to use multiple git configs on one computer. *freeCodeCamp* (2021).
25. Connolly, K. Setting up multiple SSH keys on one computer. *Medium* (2022).